МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение
высшего образования
«Казанский национальный исследовательский технический университет
им. А.Н. Туполева-КАИ»
(КНИТУ-КАИ)

ИКТЗИ

(наименование института (факультета), филиала)

АСОИУ

(наименование кафедры)

Проектная работа

по дисциплине: Большие данные

на тему: Study RAG — мультиагентная разметка учебных материалов и поиск по ним с ответами со ссылками на источник

Обучающиеся:	4307	Никулин Д.Э.
	(номер группы)	(Ф.И.О.)
	4307	Тугулева К.С.
	(номер группы)	(Ф.И.О.)
Руководитель:		Мокшин В.В.
		(Ф.И.О.)

Казань 2026

Содержание

Введение ... 4
Цель и задачи ... 4
Требования к проекту ... 5
1. Анализ предметной области ... 7
1.1 Данные и что с ними не так ... 7
1.2 Сравнение с существующими решениями ... 7
2. Архитектура и технологии ... 9
2.1 Состав системы ... 9
2.2 Технологии ... 10
2.3 Структура репозитория ... 10
2.4 Как укладываемся в бесплатные тарифы ... 11
3. Модель данных ... 12
3.1 Таблицы и связи ... 12
3.2 Разграничение доступа и лимиты ... 12
4. Загрузка датасета ... 14
5. Мультиагентная разметка ... 15
5.1 Правила: архитектор, критик, человек ... 15
5.2 Роли и очередь ... 15
5.3 Разметка основного датасета ... 16
5.4 Разметка в приложении ... 16
6. Поиск и ответ (RAG) ... 18
6.1 Индексация ... 18
6.2 Путь вопроса ... 18
6.3 Интерфейс чата ... 19
7. Оценка качества ... 20
7.1 Согласие разметчиков ... 20
7.2 Согласие с серебряным набором ... 20
7.3 Качество поиска ... 22
7.4 Ограничения оценки ... 23
8. Пользовательский интерфейс ... 24
8.1 Дизайн ... 24
8.2 Экраны ... 24
8.3 Text-to-UI: экраны из v0 ... 31
9. Тестирование и CI/CD ... 33

9.1 Результаты проверок 33
9.2 Конвейер GitHub Actions 33
9.3 Ручная проверка 33
10. Развёртывание 35
10.1 Публичный стенд 35
10.2 Настройки 35
10.3 Docker 35
11. Работа с ИИ-агентами 36
11.1 Инструменты и ход работы 36
11.2 Что нашли проверки 36
11.3 Выводы 37
12. Ограничения и возможное развитие 38
12.1 Что не проверено 38
12.2 Известные ограничения 38
12.3 Что можно улучшить 39
Заключение 40
Список литературы 41

Введение

За пять семестров у группы накопились сотни файлов: лекции, методички, задания, отчёты, билеты. Они лежат в Blackboard, в чатах Telegram и в загрузках у разных людей. Найти, где преподаватель объяснял конкретную тему, трудно: поиск по имени файла не помогает, а общий чат-бот отвечает «из головы» и не показывает, откуда взят ответ.

Study RAG решает эту задачу в два шага. Сначала приложение размечает загруженный набор документов: агент-архитектор предлагает схему полей и правила, человек их утверждает, два агента-разметчика независимо размечают корпус, арбитр решает споры, сомнительное уходит человеку. Потом по размеченному корпусу работает RAG (retrieval-augmented generation, генерация с опорой на найденные фрагменты): ответ приходит со сносками на документ и номер страницы или слайда, а поля разметки становятся фильтрами поиска.

Первый датасет — материалы группы КИ за I–V семестры: 572 документа, 5 787 страниц, слайдов и разделов, 11 082 фрагмента в поисковом индексе. Приложение развёрнуто по адресу https://study-rag-livid.vercel.app, войти можно без регистрации кнопкой «Войти как проверяющий». Код открыт: https://github.com/fA1N-n/study-rag.

Работу поделили так. Тугулева К.С. собрала материалы из трёх источников и перевела их в Markdown с нумерацией страниц и слайдов (через агента Codex по инструкции `docs/data-collection.md`). Никулин Д.Э. спроектировал и написал приложение, провёл разметку и оценку качества; код писался с ИИ-агентом Claude Code, это описано в разделе 11.

Цель и задачи

Цель: сделать веб-приложение, которое превращает неразмеченный набор учебных документов в размеченный корпус с поиском и ответами со ссылками, и проверить на наших данных, помогает ли разметка поиску.

Для этого нужно было решить следующие задачи.

– Разобрать собранные данные и понять, чего в них не хватает для поиска.
– Спроектировать базу данных под произвольную схему разметки, очередь фоновых задач и разграничение доступа.
– Сделать загрузку документов с разбором на страницы и слайды, удалением дублей и маскированием персональных данных.
– Реализовать генерацию правил разметки агентами, утверждение человеком и мультиагентную разметку батчами с арбитром и очередью проверки.
– Построить гибридный поиск (полнотекстовый и векторный) с переписыванием запроса, автофильтрами, rerank, честным отказом и сносками.
– Оценить качество разметки и поиска и честно описать ограничения оценки.
– Настроить автоматические проверки, развернуть приложение на бесплатных тарифах и подготовить документацию.

Требования к проекту

Проект сдаётся по двум курсам: проектная работа по дисциплине «Большие данные» и итоговый проект OTUS «Разработка веб-приложения с AI-агентами». Требования у них общие; в таблице 1 для каждого указано, где оно выполнено. Отдельно мы поставили себе ограничение: не тратить денег, поэтому все сервисы и модели — на бесплатных тарифах.

Таблица 1. Требования и их реализация

Требование	**Что сделано**
Не меньше трёх экранов	11 экранов: вход, датасеты, обзор, загрузка, документы, документ, правила, разметка, проверка, чат, история
Адаптивная вёрстка	Tailwind CSS 4; на телефоне колонки чата и доски становятся вкладками; проверено на ширине 375 px в светлой и тёмной теме
Формы с валидацией	zod на клиенте и сервере, CHECK-ограничения в БД: вход, датасет, метки документа, загрузка, вопрос

Требование	Что сделано
Состояния загрузки, ошибок, пустые	индикаторы «Входим…», «Сохраняем…», «…» на кнопке вопроса, этапы загрузки; сообщения об ошибке с кнопкой «Повторить»; «Пока пусто» в пустых списках
Postgres, не меньше трёх связанных таблиц	9 таблиц с внешними ключами (раздел 3), 8 миграций
CRUD в интерфейсе	датасеты (создать, переименовать, удалить); документы (загрузить, править метки, удалить); правила (сгенерировать, править, утвердить)
Аутентификация	Supabase Auth, регистрация закрыта, демо-вход для проверяющего с лимитами, RLS
Внешний API	бесплатные LLM OpenRouter с запасной моделью Cloudflare Workers AI; эмбеддинги Cloudflare bge-m3
Загрузка файлов	zip, md, txt, pdf, docx из браузера в Supabase Storage, разбор на страницы, слайды и разделы
Поиск и фильтры	гибрид FTS + вектор + RRF; фильтры чата строятся из схемы разметки; таблица документов с фильтрами
Деплой	Vercel: https://study-rag-livid.vercel.app
CI/CD	GitHub Actions: lint, проверка типов, тесты, сборка; автодеплой Vercel из ветки main
Docker	Dockerfile с output: "standalone"; standalone-сборка проверена, сборка образа — нет (Docker на машине не установлен)
Тесты	227 автотестов (Vitest), из них 25 SQL/RLS-проверок в PGlite; 3 smoke-теста по живому серверу
Документация и журнал работы с ИИ	README, ARCHITECTURE.md, docs/ai-log.md, docs/eval.md
Text-to-UI	экраны чата и доски разметки сгенерированы v0 по текстовому описанию (раздел 8.3)

1. Анализ предметной области

1.1 Данные и что с ними не так

Материалы собирались из Blackboard, экспорта чатов Telegram и личных папок. Агент Codex перевёл их в Markdown: каждый документ — отдельный файл с заголовками «Страница N» или «Слайд N» и блоком метаданных (предмет, семестр, преподаватель, тип). В архиве от 7 ноября 2026 года — 572 документа объёмом 22 МБ.

Разбор архива показал, что использовать его как есть нельзя (таблица 2).

Таблица 2. Пробелы в исходных данных

Что нашли	Масштаб	Чем мешает
Нет предмета	236 документов (41 %)	нельзя отфильтровать поиск по дисциплине
Нет преподавателя	240 документов	нельзя спросить «что говорил преподаватель N»
Тип поставлен по умолчанию	«задание» у 244, «другое» у 131	отчёты студентов не отличить от методичек
Чужие персональные данные	около 248 документов с титульными листами	ФИО, группы, телефоны нельзя отправлять в LLM и показывать на демо
Битые имена файлов	104 имени с символами «□»	имя файла не помогает понять, что внутри
Картинки без описаний	все 5 014	часть смысла слайдов потеряна
Контрольные вопросы не проверены	64 вопроса, все помечены «оставить»	для оценки поиска нужна чистка

Отсюда главная идея проекта. Если разметку надо делать в любом случае, пусть её делает приложение, а человек только утверждает правила и разбирает сомнительное. Тогда тот же конвейер подойдёт для любого другого набора: схема полей не зашита в код, её предлагает агент под конкретные документы.

1.2 Сравнение с существующими решениями

Для сравнения взяты три продукта: NotebookLM (Google) и проекты ChatGPT (OpenAI) — ближайшие аналоги для вопросов по своим документам,

и Label Studio — популярный открытый инструмент разметки данных. Сравнение составлено по открытым описаниям продуктов (справка и сайты разработчиков) на ноябрь 2026 года. На наших данных эти продукты мы не запускали: материалы содержат чужие персональные данные, а сравнительный прогон не поместился в сроки. Поэтому таблица 3 сравнивает возможности, а не качество ответов.

Таблица 3. Сравнение Study RAG с другими решениями

Возможность	**NotebookLM**	**Проекты ChatGPT**	**Label Studio**	**Study RAG**
Вопросы по своим документам	да	да	нет	да
Ссылка на место в источнике	да, на фрагмент	на файл, место — не всегда	—	да, на документ и страницу/слайд
Схема разметки корпуса	нет	нет	задаёт человек	предлагает агент, утверждает человек
Авторазметка и спорные случаи	нет	нет	предразметка моделью, проверка людьми	два разметчика, арбитр, очередь человеку
Фильтры по полям разметки	выбор источников вручную	нет	да, в интерфейсе разметки	да, в поиске и автоматически из вопроса
Маскирование ПДн до LLM	нет	нет	нет	да, при загрузке
Открытый код, развёртывание у себя	нет	нет	да (Community)	да
Стоимость	бесплатно с лимитами, есть платный план	бесплатно с лимитами, есть подписка	Community бесплатно	0 ₽ на сервисы и модели приложения (бесплатные тарифы)

Готовые сервисы сильнее в главном: у NotebookLM и ChatGPT платные модели, ответы быстрее и, скорее всего, точнее наших бесплатных. Но они отвечают по документам «как есть» и не наводят порядок в самом корпусе. Label Studio, наоборот, умеет разметку, но не умеет искать и отвечать. Study RAG соединяет эти две части и делает их на одной схеме полей.

2. Архитектура и технологии

2.1 Состав системы

Приложение — один проект Next.js на Vercel. Отдельного бэкенда нет: серверная логика живёт в обработчиках маршрутов (Route Handlers) и серверных компонентах. Данные, вход и файлы — в Supabase. Модели вызываются по HTTP из серверного кода (рис. 1).

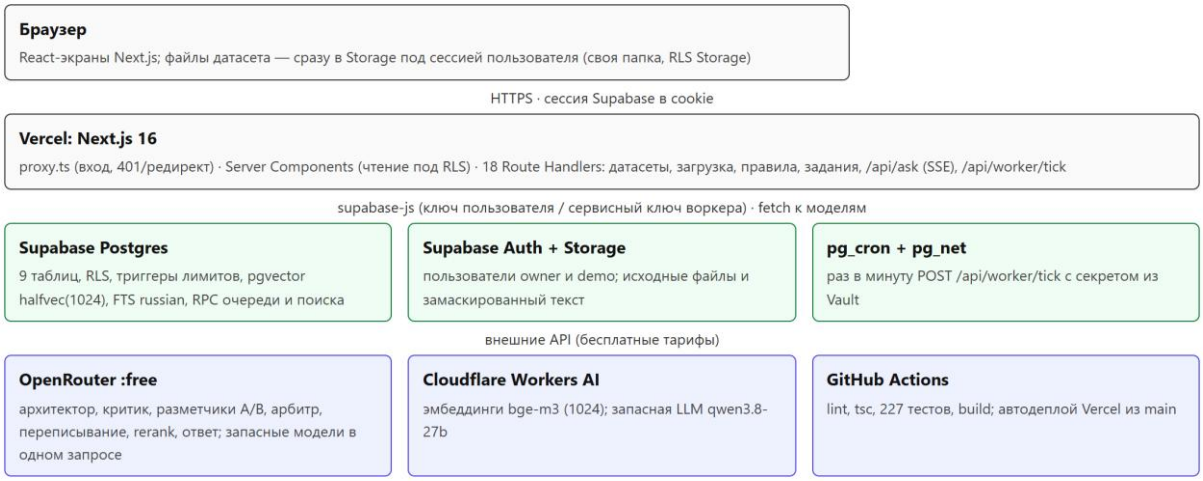


Рис. 1. Компоненты Study RAG и внешние сервисы

Долгая работа (разметка батчей, индексация) не помещается в один HTTP-запрос: у бесплатного Vercel функция живёт до 300 секунд. Поэтому задачи лежат в очереди в Postgres, а расписание pg_cron раз в минуту, если в очереди есть батчи или идёт индексация, вызывает `/api/worker/tick` с секретом из Supabase Vault. Тик забирает работу с арендой (`FOR UPDATE SKIP LOCKED`), выполняет её в пределах бюджета времени и отпускает. Кнопки в интерфейсе запускают первый тик сразу, не дожидаясь расписания.

Общий путь данных показан на рис. 2: загрузка, правила, разметка, индекс, ответ.



Рис. 2. Конвейер: от загрузки до ответа со сносками

2.2 Технологии

Версии взяты из package.json (таблица 4).

Таблица 4. Технологический стек

Слой	Технология	Зачем
Веб	Next.js 16.4.5 (App Router), React 19.3, TypeScript 5, Tailwind CSS 4	экраны, API, серверный рендеринг под сессией пользователя
Валидация	zod 4.6	одни схемы для форм, API и меток агентов
БД и вход	Supabase: Postgres + pgvector (halfvec), FTS russian, Auth, Storage, pg_cron, pg_net, Vault	данные, RLS, очередь, поиск
Разбор файлов	fflate 0.8 (zip), unpdf 2.1 (pdf), mammoth 1.13 (docx)	текстовый слой без внешних сервисов
LLM	OpenRouter, бесплатные модели `:free`; запасная — Cloudflare `@cf/qwen/qwen3.8-27b`	архитектор, критик, разметчики, арбитр, переписывание, rerank, ответ
Эмбеддинги	Cloudflare Workers AI `@cf/baai/bge-m3`, 1024 измерения	векторный поиск по-русски
Тесты	Vitest 5.1, PGlite 0.5 + pgvector	unit-тесты и SQL/RLS в памяти без Docker
CI/CD и хостинг	GitHub Actions, Vercel Hobby, Dockerfile (Node 24)	проверки на push в main и pull request, автодеплой

2.3 Структура репозитория

```
src/app/              экраны (App Router) и API (app/api/*)
```

```
src/lib/ingest/     загрузка: разбор, единицы, sha256, маскирование ПДн
src/lib/labeling/   схема, валидатор, батчи, слияние A/B/арбитр, каппа
src/lib/rag/        чанки, эмбеддинги, индексатор, поиск и ответ
src/lib/llm.ts      вызов моделей: цепочка OpenRouter → Cloudflare
src/lib/worker.ts   тик воркера: батчи разметки и индексация
supabase/           8 миграций и SQL/RLS-тесты
scripts/            импорт, индексация, офлайн-разметка, оценка
docs/               ТЗ, план, дизайн, метрики, журнал работы с ИИ
smoke/              smoke-тесты по живому серверу
```

2.4 Как укладываемся в бесплатные тарифы

Каждый бесплатный тариф ограничен по-своему; как мы в них укладываемся, показано в таблице 5.

Таблица 5. Бесплатные лимиты и решения

Ресурс	**Лимит**	**Решение**
OpenRouter `:free`	50 запросов в день, модели перегружены	один запрос на батч для каждого разметчика; список запасных моделей в одном запросе; затем Cloudflare
Cloudflare Workers AI	10 000 нейронов в день	эмбеддинги корпуса — один раз; LLM — только запасной путь
Vercel Hobby	функция до 300 с	короткие тики с бюджетом времени; большой датасет индексируется скриптом
Supabase Free	500 МБ БД	векторы в halfvec (вдвое меньше vector)

3. Модель данных

3.1 Таблицы и связи

Схема БД описана в восьми миграциях `supabase/migrations/`. Центральная таблица — `datasets`; документы, правила, задания разметки, фрагменты и вопросы ссылаются на неё и удаляются вместе с ней (таблица 6).

Таблица 6. Таблицы базы данных

Таблица	Что хранит	Ключевые поля и ограничения
profiles	роль пользователя	role: owner или demo
datasets	набор документов	owner_id, is_demo, status (empty → … → ready), active_ruleset_id, stats (jsonb)
documents	документ набора	sha256 (unique в датасете), units, preview, pii_flag, labels (jsonb + GIN), label_status
rulesets	версия схемы и правил	fields (jsonb), rules, examples, critique, status: draft/approved/archived, unique (dataset_id, version)
label_jobs	задание разметки	ruleset_id, счётчики батчей, токены; одно активное задание на датасет
batches	батч задания	document_ids, status, attempts, locked_until (аренда)
labels	каждый голос	agent: A, B, referee, human; model, values, confidence, rationale
chunks	фрагмент для поиска	unit_no, header, content, embedding halfvec(1024) + HNSW, tsv (FTS russian) + GIN
questions	история вопросов	question, rewritten_query, filters, answer, sources, refused, feedback (−1/1), latency_ms

Метки документа хранятся в jsonb, а не в отдельных колонках. Так у каждого датасета может быть своя схема полей, а фильтр поиска записывается одним условием `labels @> {...}` по GIN-индексу. Каждый голос агента и человека сохраняется в `labels` отдельной строкой: по ним считается согласие разметчиков и видно, кто и почему поставил метку.

3.2 Разграничение доступа и лимиты

Доступ проверяет сама БД через RLS (Row Level Security). Две функции решают всё: `can_read_dataset` — владелец или демо-датасет, `can_write_dataset` — только владелец. Вопросы в истории видны только их автору. Батчи, голоса агентов и фрагменты пишет только сервисный ключ

воркера; право вызывать функции очереди отозвано у ролей `anon` и `authenticated`.

Проверяющий входит без пароля, поэтому для него действуют лимиты. Триггеры в БД держат не больше 30 документов в датасете (с `pg_advisory_xact_lock`, чтобы параллельные загрузки не обошли лимит) и не больше 30 вопросов в сутки; API разрешает не больше трёх запусков разметки в сутки. Разметку нельзя запустить по неутверждённым правилам — это тоже проверяет триггер, а не только интерфейс.

4. Загрузка датасета

Пользователь выбирает файлы или zip-архив. Браузер кладёт их в Supabase Storage в папку пользователя (политики Storage не дают писать в чужую папку), затем вызывает `/api/datasets/[id]/ingest`. Сервер читает файлы и разбирает их ядром `src/lib/ingest/`:

1. распаковывает zip и чинит имена в кодировке CP866, прочитанные как CP437;
2. извлекает текст из md, txt, pdf (текстовый слой) и docx;
3. делит документ на единицы «Страница N», «Слайд N» или «Раздел»;
4. считает sha256 и пропускает дубли (уникальный индекс в БД страхует от гонок);
5. маскирует персональные данные в тексте, подписях, имени файла и пути;
6. сохраняет документ уже замаскированным, поэтому ни LLM, ни демо-пользователь оригинальных ФИО не видят.

Маскирование ловит ФИО (в том числе с отчеством и заглавными буквами), телефоны и почты, номера документов после слов «паспорт», «билет», «СНИЛС», фамилии в именах файлов. Преподавателей из схемы разметки оставляет: без их фамилий не работает фильтр «преподаватель». Наш архив разбирается примерно за 2 секунды, поэтому загрузка идёт синхронно в одном запросе, без очереди.

Ядро загрузки написано через TDD с двумя независимыми критиками: 22 критерия (`docs/ingest-core.md`), тесты, критик тестов, код, финальный критик с прогоном на всех 572 документах. Итог — 76 тестов и 16 исправленных дефектов (раздел 11.2). Результат на нашем архиве: 572 документа, 5 787 единиц, персональные данные найдены и замаскированы в 410 документах. Это больше, чем 248 документов с титульными листами из раздела 1.1: маска срабатывает и на ФИО, телефоны и почты в тексте, подписях и именах файлов.

5. Мультиагентная разметка

5.1 Правила: архитектор, критик, человек

Агент-архитектор получает стратифицированную выборку документов (разные папки, источники, размеры) и предлагает схему полей — тип, допустимые значения, обязательность, роль фильтра — и письменные правила с примерами и порядком решения. Критик применяет правила как разметчик к другой выборке и отмечает, где они противоречат друг другу или где два добросовестных разметчика разойдутся. Архитектор правит. Человек смотрит результат на экране «Правила», при необходимости правит и утверждает версию (рис. 11).

Для основного датасета правила v1 сделаны офлайн агентами Claude Code: выборка 30 документов, критик на других 20 нашёл 16 замечаний, все приняты. В схеме 6 полей: предмет (35 значений и «другое»), семестр, преподаватель, тип документа (12 значений), название, есть ли персональные данные. В приложении тот же путь проверен на тестовом датасете из 8 документов бесплатной моделью.

5.2 Роли и очередь

Роли участников разметки сведены в таблице 7.

Таблица 7. Роли в разметке

Роль	**Кто**	**Что делает**
Архитектор и критик	LLM	схема и правила по выборкам документов
Человек	владелец датасета	утверждает правила, решает спорное в очереди проверки
Координатор	код	режет документы на батчи (до 10 документов и ~12 тыс. токенов), ставит в очередь, повторяет упавшие до 3 раз
Разметчики A и B	две разные LLM	независимо размечают батч: значения, уверенность 0–1, цитата-обоснование
Валидатор	код (zod)	значения из словаря, обязательные поля, согласованность семестра с предметом
Арбитр	LLM	только спорные документы: решает по правилам и обоим обоснованиям

Разметчики видят начало документа (до ~3 страниц), уже без персональных данных. Если A и B совпали по ключевым полям и оба уверены (не ниже 0,7), документ получает статус agreed. Если разошлись, вызывается арбитр; уверенное решение — refereed, неуверенное или низкая уверенность разметчиков — review, то есть очередь человеку. Чтобы уложиться в 50 бесплатных запросов в день, каждый разметчик получает батч одним запросом, а арбитр — все спорные документы батча одним запросом.

Ход разметки виден на доске (рис. 12): число размеченных, доля авто, очередь человеку, батчи, модели агентов, итог по статусам, каппа. В очереди проверки человек видит документ и голоса агентов с цитатами и решает клавишами 1, 2, 3 (рис. 13).

5.3 Разметка основного датасета

572 документа размечены офлайн агентами по правилам v1 и импортированы в БД тем же форматом, что пишет воркер. Разметчик A — Claude Sonnet (29 субагентов, по батчу на каждого). Разметчик B — Codex (батчи 1–18) и Claude Haiku (батчи 19–29). Арбитр — Claude Opus (5 субагентов по 69 спорным документам). Итог: agreed — 434 документа (75,9 %), refereed — 39 (6,8 %), review — 99 (17,3 %). Арбитр уверенно решил 39 из 69 споров.

Почему офлайн: 572 документа × 2 разметчика + арбитр не помещаются в 50 бесплатных запросов в день, а платить мы не стали. Агенты Claude Code и Codex работали по подписке, которая у нас уже была. Очередь проверки из 99 документов людьми не пройдена — это ограничение (раздел 12).

5.4 Разметка в приложении

Полный путь внутри приложения проверен на тестовом датасете проверяющего «Тест: 8 документов КИ»: правила сгенерированы бесплатной моделью и утверждены в интерфейсе, разметка прошла одним батчем за 192 секунды, 27 тыс. входных и 20 тыс. выходных токенов, $0. Итог: agreed 1,

refereed 6, review 1; решение человека из очереди сохранилось с меткой human, отмена задания работает. Каппа по типу документа — 1,0, по кафедре — 0,3: правила, придуманные по 8 документам, оставили поле свободным текстом, и разметчики писали «АСОИУ» и «Кафедра АСОИУ».

6. Поиск и ответ (RAG)

6.1 Индексация

Фрагмент (чанк) — одна единица документа: страница или слайд. Длинная единица режется на части по ~2 000 символов с перекрытием 200 по границе абзаца или предложения. В начало фрагмента добавляется заголовок из меток: «Предмет · Семестр · Преподаватель · Тип · Документ · Слайд N». Заголовок попадает и в эмбеддинг, и в полнотекстовый индекс, поэтому вопрос «что было у Мокшина про…» находит нужный предмет, даже если фамилии нет в тексте слайда. Эмбеддинги считает Cloudflare bge-m3 пачками по 50. Индекс основного датасета — 11 082 фрагмента.

6.2 Путь вопроса

Вопрос проходит девять шагов в `/api/ask`, ответ идёт стримом (Server-Sent Events):

1. вопрос записывается в историю до вызова модели, поэтому лимит проверяющего срабатывает сразу;
2. быстрая модель переписывает запрос и предлагает автофильтры; значения берутся только из словаря схемы, выдумать предмет модель не может;
3. векторный поиск (top-40) и полнотекстовый по-русски (top-40) объединяются методом RRF (Reciprocal Rank Fusion) в 30 кандидатов;
4. если автофильтр дал пустую выдачу, поиск повторяется без него, и интерфейс об этом сообщает;
5. модель оценивает 20 кандидатов по шкале 0–10 (rerank);
6. остаются до 6 фрагментов с оценкой не ниже 4; если таких нет — честный отказ со списком ближайших документов;
7. к фрагментам добавляются соседние страницы ±1;
8. модель пишет ответ со сносками [n] на документ и страницу;

9. ответ и источники сохраняются в историю, пользователь может поставить 👍 или 👎 .

Нейронный reranker Cloudflare `bge-reranker-base` мы проверили и отказались: на русском он поставил правильный фрагмент последним. Поэтому rerank делает та же бесплатная LLM.

6.3 Интерфейс чата

Слева — фильтры, построенные из схемы разметки датасета (для enum — список, для чисел — значения, для флагов — да/нет), и свои вопросы. В центре — ответ стримом, переписанный запрос и снимаемые чипы автофильтров. Справа — источники с оценкой rerank и панель документа с подсвеченной страницей (рис. 14). Ответ бесплатной моделью идёт 30–100 секунд.

7. Оценка качества

Метрики пересчитываются скриптами `scripts/eval/labeling-eval.ts` и `scripts/eval/search-eval.ts`, полные таблицы — в `docs/eval.md`. У оценки есть ограничения, они названы рядом с цифрами и сведены в разделе 7.4.

7.1 Согласие разметчиков

Каппа Коэна показывает согласие A и B с поправкой на случайное совпадение (таблица 8).

Таблица 8. Каппа Коэна между разметчиками A и B, 572 документа

Поле	Все 572	B = Codex	B = Haiku
Предмет	0,959	0,941	0,990
Семестр	0,934	0,905	0,982
Преподаватель	0,962	0,945	0,989
Тип документа	0,907	0,872	0,968
Есть ПДн	0,965	0,946	1,000

С Haiku согласие выше, чем с Codex. Это не значит, что Haiku точнее: A и Haiku — модели одного семейства Claude, их общие ошибки каппа не видит. Похожее смещение может быть у арбитра: он встал на сторону A в 102 из 116 спорных полей, а A и арбитр — тоже модели Claude.

7.2 Согласие с серебряным набором

Золотого набора, размеченного людьми, у нас нет: на 80 документов не хватило времени. Вместо него сделан **серебряный набор**: те же 80 документов из списка кандидатов (31 — покрытие предметов, 6 — редкие типы, 43 — низкая уверенность разметчиков) разметили 8 независимых агентов Claude Opus по полному тексту документа (до 30 000 символов) по тем же правилам v1, не видя голосов A и B. Поэтому таблица 9 показывает согласие с более сильной моделью, которая видит больше текста, а не истинную точность.

Сравнение — без учёта регистра, ё/е и пробелов, у преподавателя — по фамилии; название не сравнивается.

Таблица 9. Согласие с серебряным набором (80 документов)

Разметчик	Предмет	Семестр	Преп.	Тип	ПДн	Все поля
A (Sonnet)	92,5 %	91,3 %	92,5 %	96,3 %	95,0 %	83,8 %
B (Codex/Haiku)	87,5 %	85,0 %	88,8 %	83,8 %	95,0 %	67,5 %
Двое + арбитр	93,8 %	92,5 %	93,8 %	96,3 %	95,0 %	83,8 %

95 % доверительный интервал Уилсона для «все поля»: у A и итога — 74–90 %, у B — 57–77 %. Разница между A и итогом лежит внутри интервала, то есть схема «двое + арбитр» по документу целиком не лучше лучшего одиночного разметчика. Выборка не случайная: 43 из 80 документов отобраны по низкой уверенности разметчиков, а у 11 текст обрезан до 30 000 символов, поэтому цифры не переносятся на весь корпус. Польза схемы видна в маршрутизации (таблица 10).

Таблица 10. Совпадение с серебряным набором по статусу маршрутизации

Статус	Документов	Все поля совпали
agreed	41	92,7 %
refereed	7	100 %
review	32	68,8 %

Документы, которые схема отправила человеку, расходятся с серебряным набором примерно в 4 раза чаще согласованных: 10 из 32 (31 %) против 3 из 41 (7 %). Выборки маленькие, а эталон — модель, поэтому это признак, а не доказательство того, что ручная проверка 17 % корпуса ловит основную часть ошибок.

Ещё 19 документов проверил человек. По документу целиком с ним совпали 31,6 % у A, B и итога (6 из 19, интервал 15–54 %) и 36,8 % у серебряного набора (7 из 19, интервал 19–59 %); по отдельным полям — от 57,9 до 89,5 %. Сильнее всего расходится поле «есть ПДн»: человек и правила

v1 понимают его по-разному. Выборка маленькая, но вывод ясен: правила v1 нужно уточнить, а серебряный набор нельзя выдавать за эталон.

7.3 Качество поиска

Для оценки взяты контрольные вопросы `questions.csv`: из 64 после чистки осталось 62 — 54 вопроса с ответом (у 53 ожидаемый документ есть в БД) и 8 вопросов без ответа в корпусе. Попадание засчитывается, если ожидаемый документ среди первых пяти документов выдачи (hit@5); MRR@10 — средний обратный ранг. Метрика строгая: если на вопрос отвечает и другой документ, это промах.

Оценка меряет поиск, а не весь путь `/api/ask`. Режим «с фильтром» берёт предмет из разметки самого вопроса в `questions.csv`, то есть фильтр заранее верный; запрос не переписывается, автофильтр не угадывается. Поэтому 83,0 % — верхняя оценка пользы фильтра. Ближе всего к рабочему пути строка с rerank — 70,0 % на 20 вопросах.

Результаты — в таблице 11.

Таблица 11. Качество поиска

Режим	**n**	**hit@5**	**MRR@10**
Гибрид без фильтров по меткам	53	73,6 %	0,544
Гибрид + верный фильтр по предмету	53	83,0 %	0,630
Подвыборка: с фильтром, без rerank	20	80,0 %	0,614
Подвыборка: с фильтром + rerank LLM (порог 4/10)	20	70,0 %	0,512

- С верным фильтром по предмету hit@5 — 83,0 % (44 из 53, интервал Уилсона 71–91 %): цель 0,8 достигнута, но в пределах погрешности. Фильтр из схемы разметки добавил 9,4 п. п. hit@5 (5 вопросов) и 0,086 MRR — это польза разметки для поиска при верно выбранном предмете.
- Rerank бесплатной моделью ухудшил выдачу на подвыборке (первые 20 вопросов, не случайные): hit@5 упал с 80 до 70 %, то есть на 2

вопроса. Ожидаемый документ выпал после rerank в 6 из 20 вопросов, в двух из них порог отсеял всю выдачу (ложный отказ).

– Без rerank система не отказывает вовсе. С ним на 8 вопросах без ответа (поиск без фильтра) выдача после порога оказалась пустой в 6, на отвечаемых — в 2 из 20 (ложный отказ). Отказ считается по пустой выдаче после rerank, ответ модели не оценивался.

Rerank оставлен ради отказов, но в нынешнем виде он режет хорошие фрагменты. Что можно попробовать: порог 3 вместо 4, rerank только для решения об отказе без пересортировки, более сильная модель.

7.4 Ограничения оценки

– Серебряный набор сделан моделью (Claude Opus), а не людьми. С ручной проверкой 19 документов он сходится на 36,8 % по документу целиком.
– Фильтр по предмету в оценке поиска взят из разметки вопроса, а не угадан автофильтром; переписывание запроса не оценивалось.
– Вопросы для оценки поиска сгенерированы моделью по документам; настоящих вопросов от студентов нет. Такие вопросы похожи на текст слайдов и могут завышать hit@5.
– «Без меток» означает «без фильтров по меткам»: заголовок фрагмента с метками есть в обоих режимах. Отдельный индекс без заголовков не строили — не хватило бесплатного лимита эмбеддингов.
– Rerank оценён на первых 20 вопросах (не случайная подвыборка) из-за лимита 50 запросов в день.
– Разметчики А и арбитр — модели одного семейства; согласие и решения арбитра могут быть смещены в пользу А.

8. Пользовательский интерфейс

8.1 Дизайн

Перед вёрсткой сделали три варианта HTML-макетов главного экрана чата и доски разметки на одинаковых выдуманных данных (рис. 3 и 4): A «Консоль» — плотный и нейтральный, сетка батчей; B «Конспект» — мягкий, чат пузырями, канбан по статусам; C «Пульт» — контрастная сетка, ответ из пронумерованных утверждений. У каждого есть светлая и тёмная тема и мобильная версия, всего 18 скриншотов (`docs/screenshots/design/`). Выбрали A по критерию «проще и быстрее сделать»; токены и классы — `docs/design.md`, `src/components/ui.ts`.

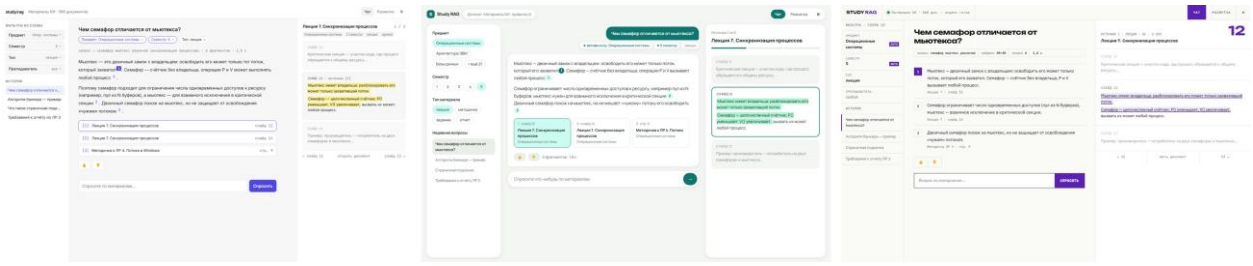


Рис. 3. Три варианта дизайна чата: A «Консоль», B «Конспект», C «Пульт»

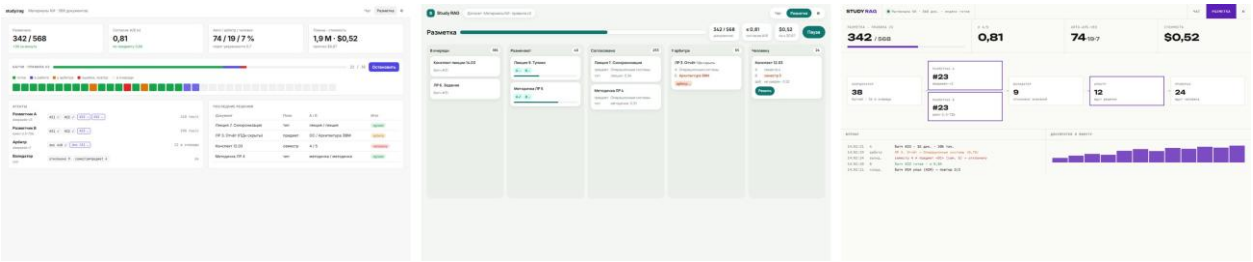


Рис. 4. Три варианта доски разметки

8.2 Экраны

Ниже — экраны рабочего приложения на проде под демо-входом (ноябрь 2026): вход (рис. 5), датасеты (рис. 6), обзор (рис. 7), загрузка (рис. 8), документы и документ (рис. 9, 10), правила, разметка и проверка (рис. 11–13), чат и источник (рис. 14, 15), история (рис. 16), мобильная версия (рис. 17).

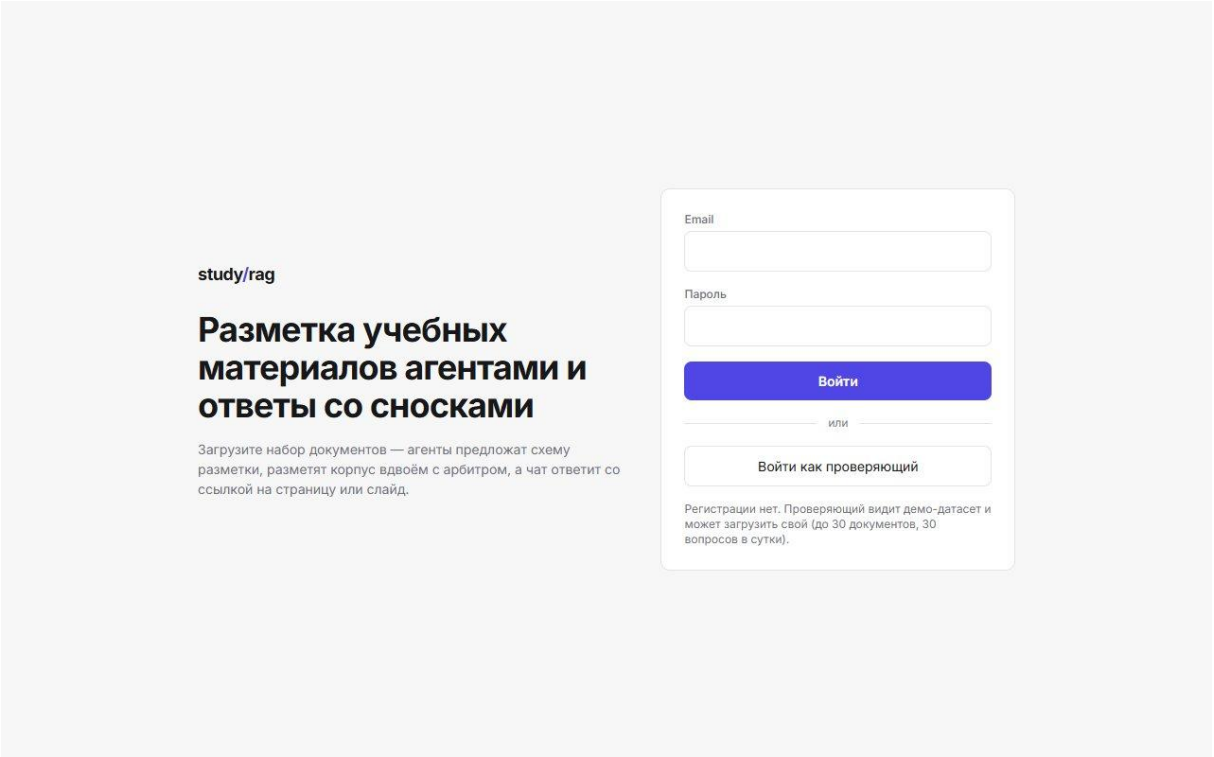


Рис. 5. Вход: кнопка «Войти как проверяющий» входит без пароля на стороне браузера

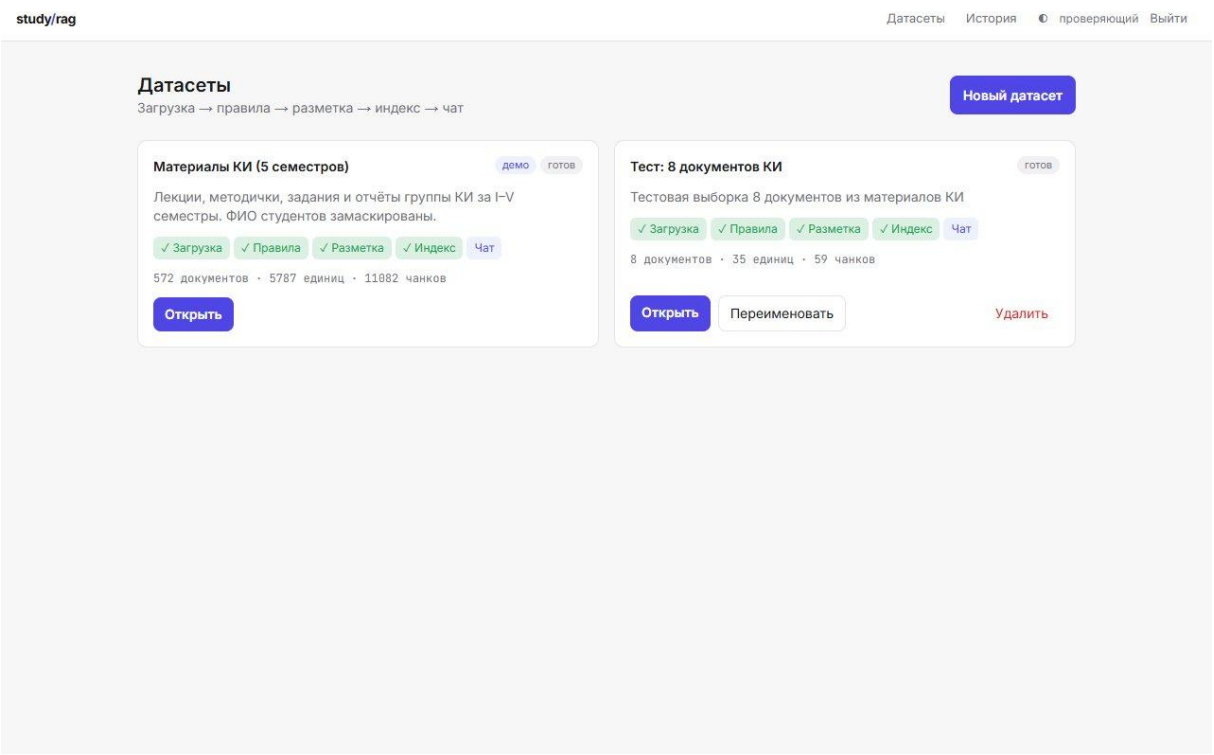


Рис. 6. Список датасетов: основной (демо, только чтение) и тестовый датасет проверяющего

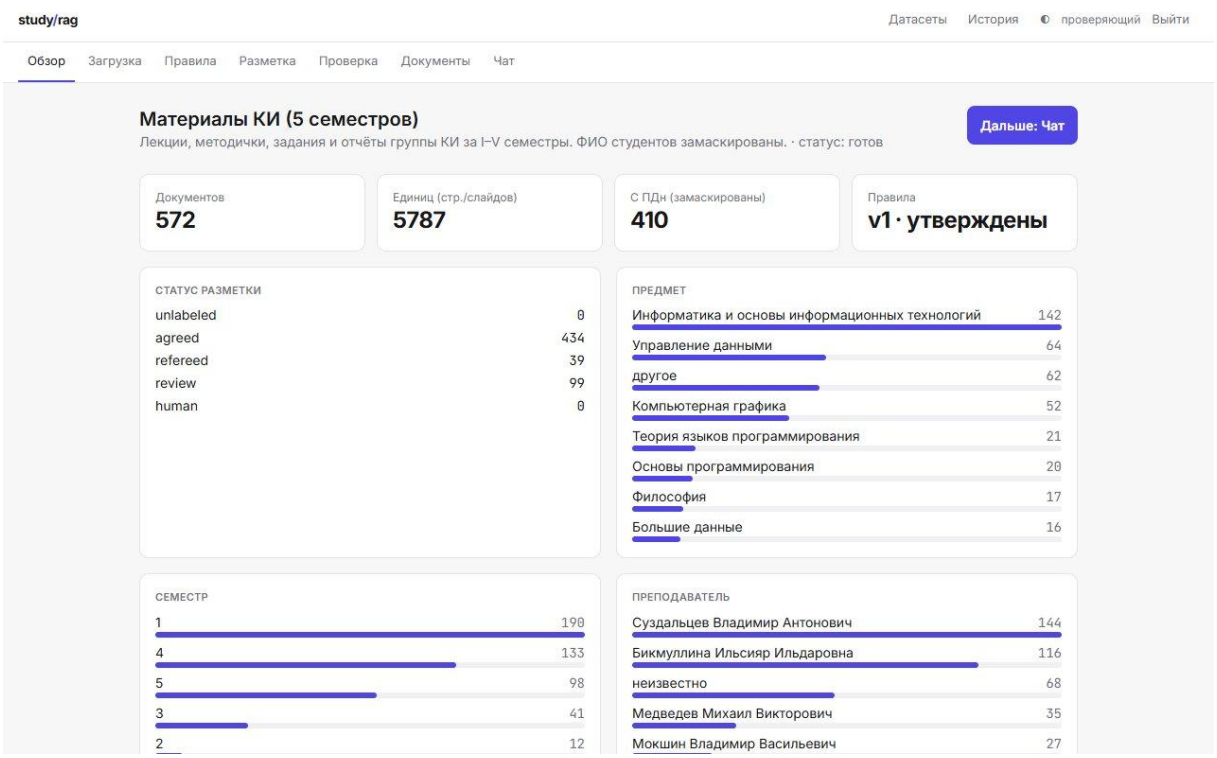


Рис. 7. Обзор датасета: документы, единицы, ПДн, статус разметки и распределение меток

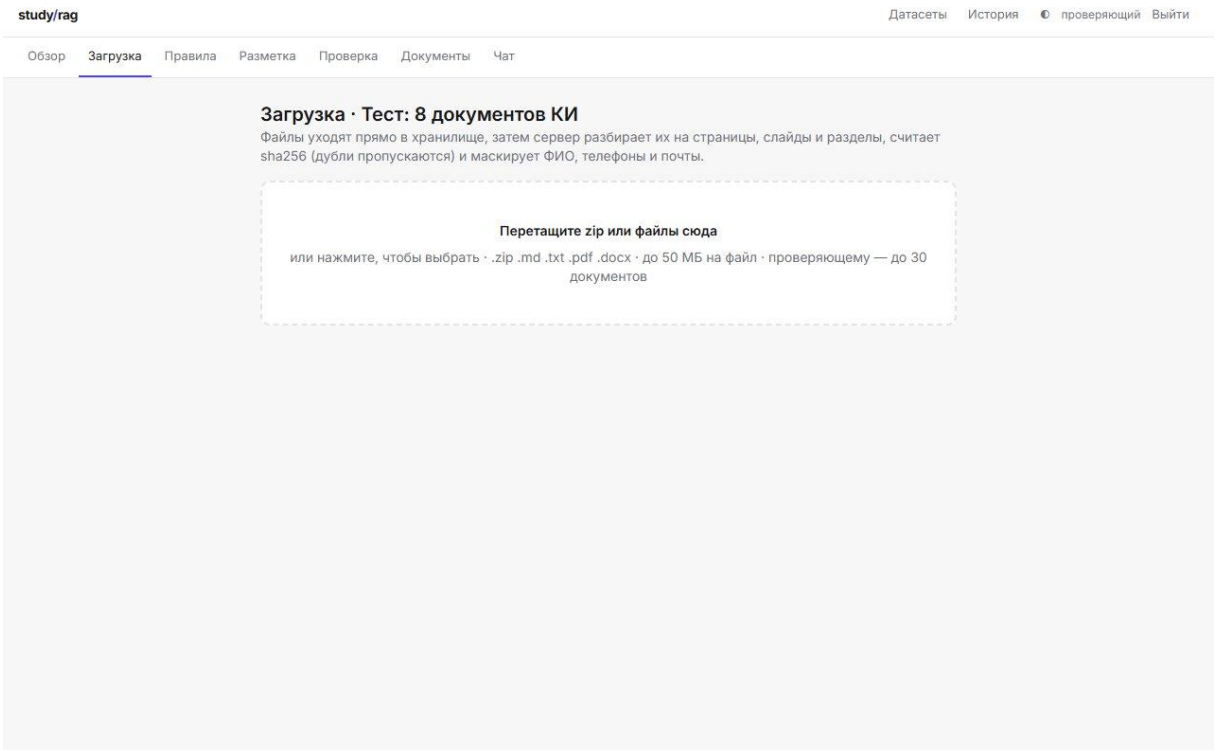


Рис. 8. Загрузка файлов в свой датасет

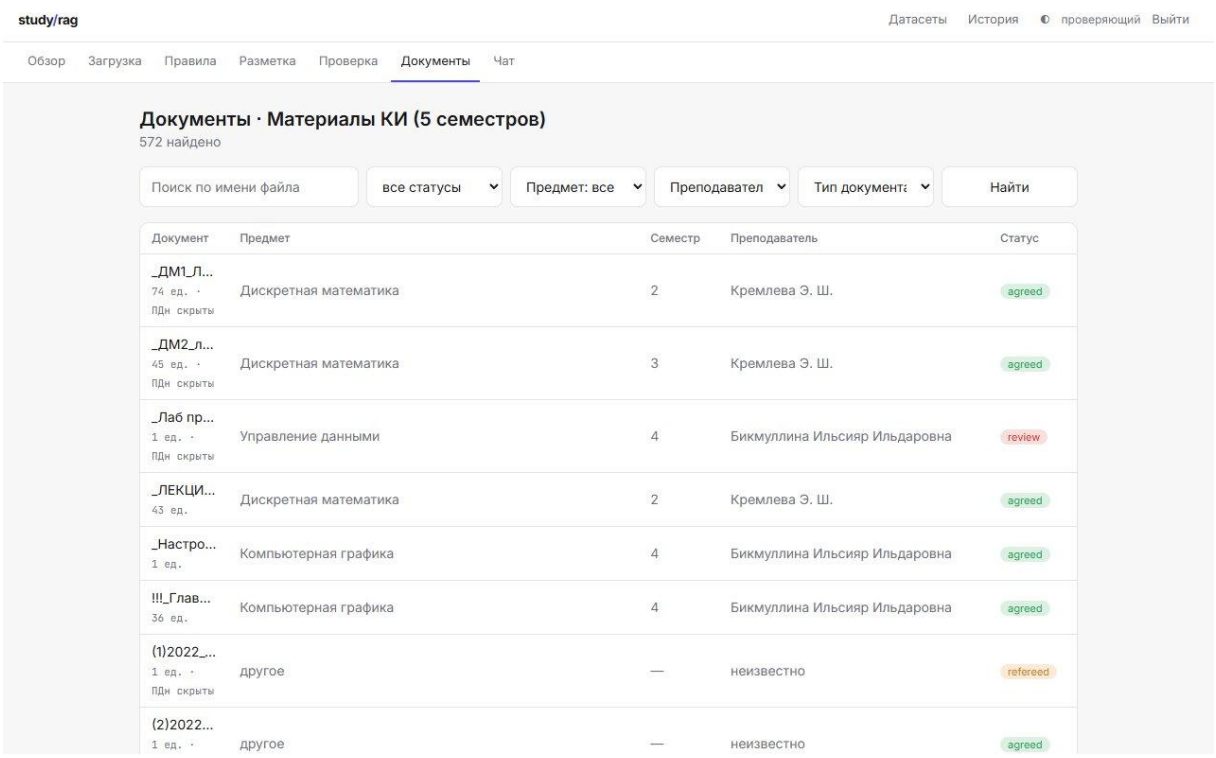


Рис. 9. Таблица документов с фильтрами по меткам

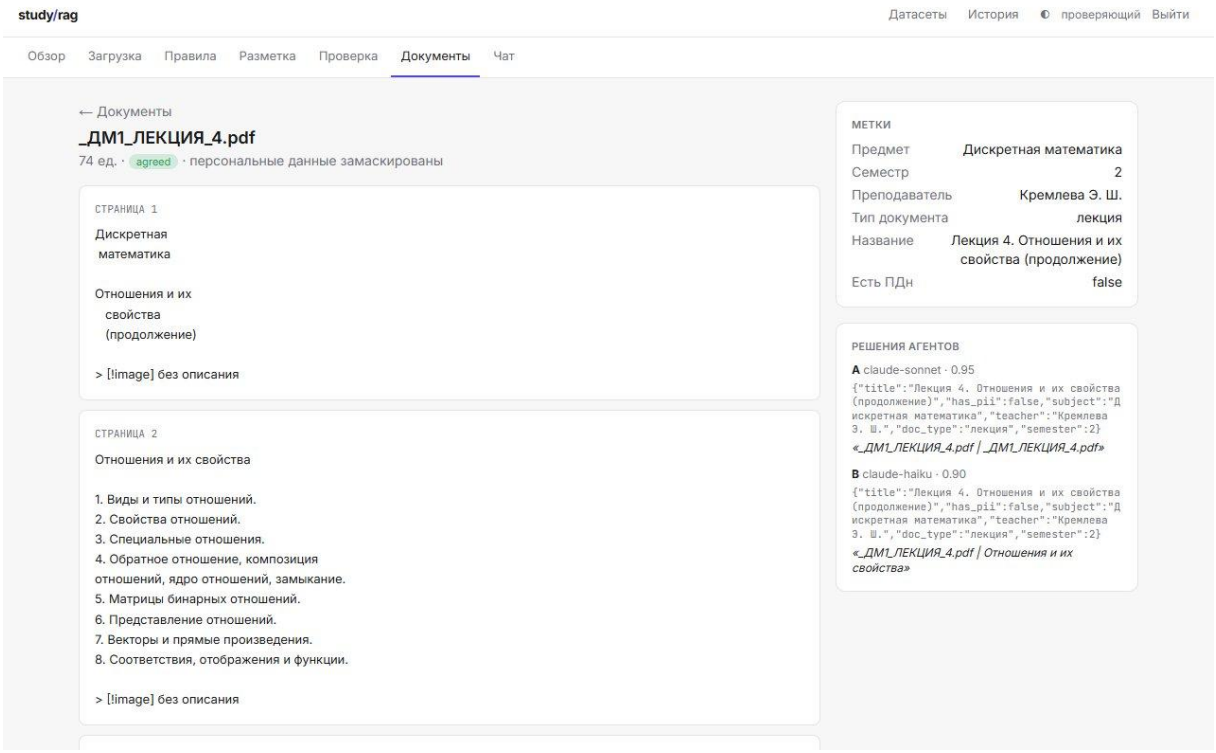


Рис. 10. Документ по страницам, метки и решения агентов с цитатами

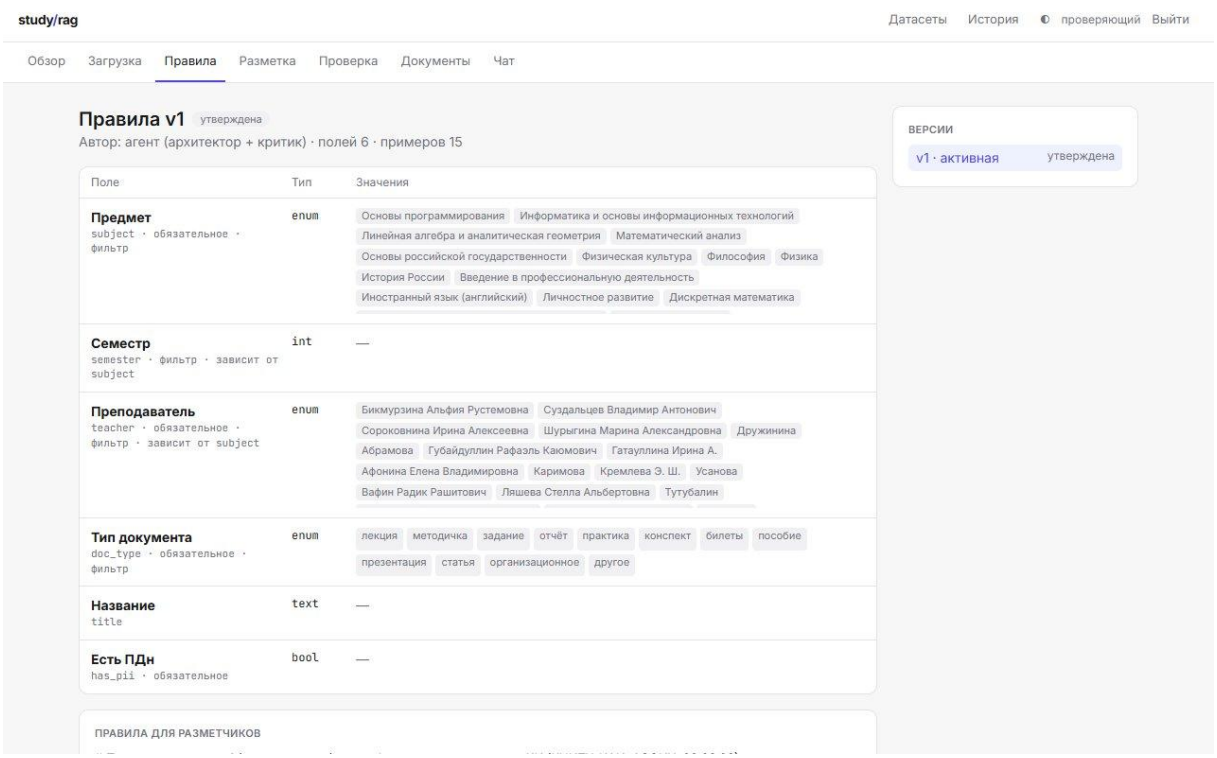


Рис. 11. Правила v1: поля схемы, значения, версии

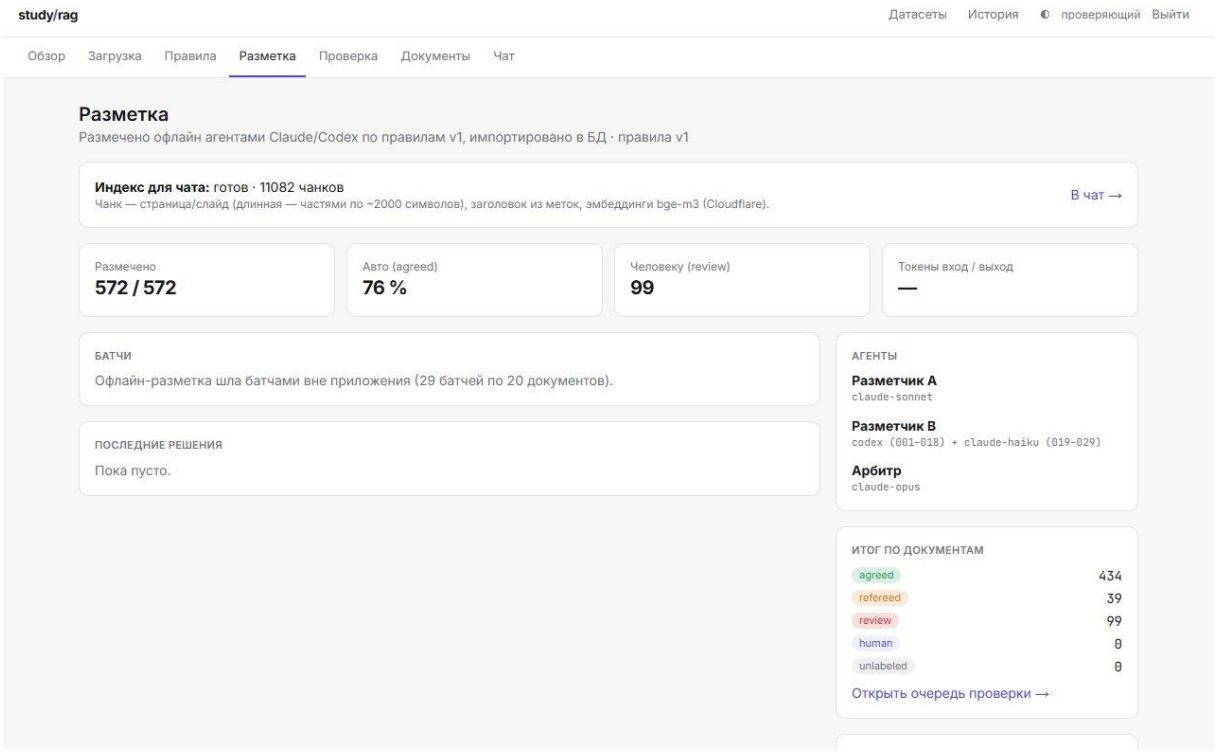


Рис. 12. Доска разметки основного датасета

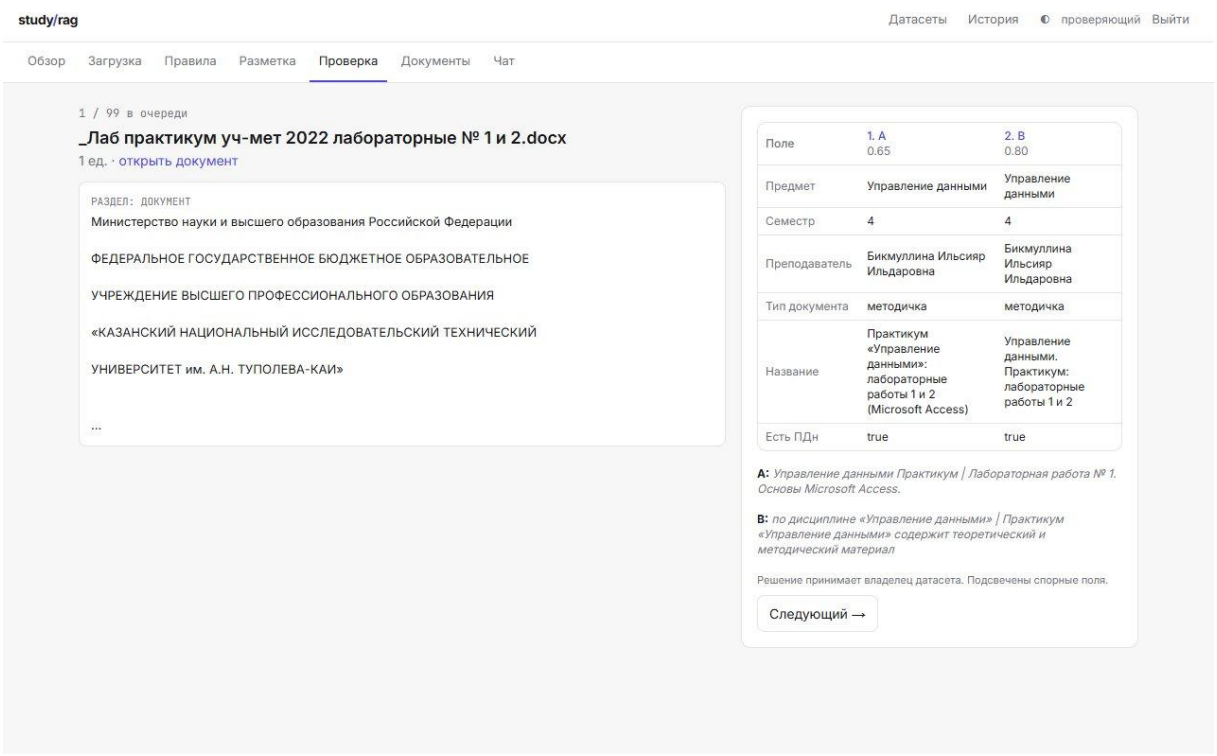


Рис. 13. Очередь проверки: документ и голоса A и B

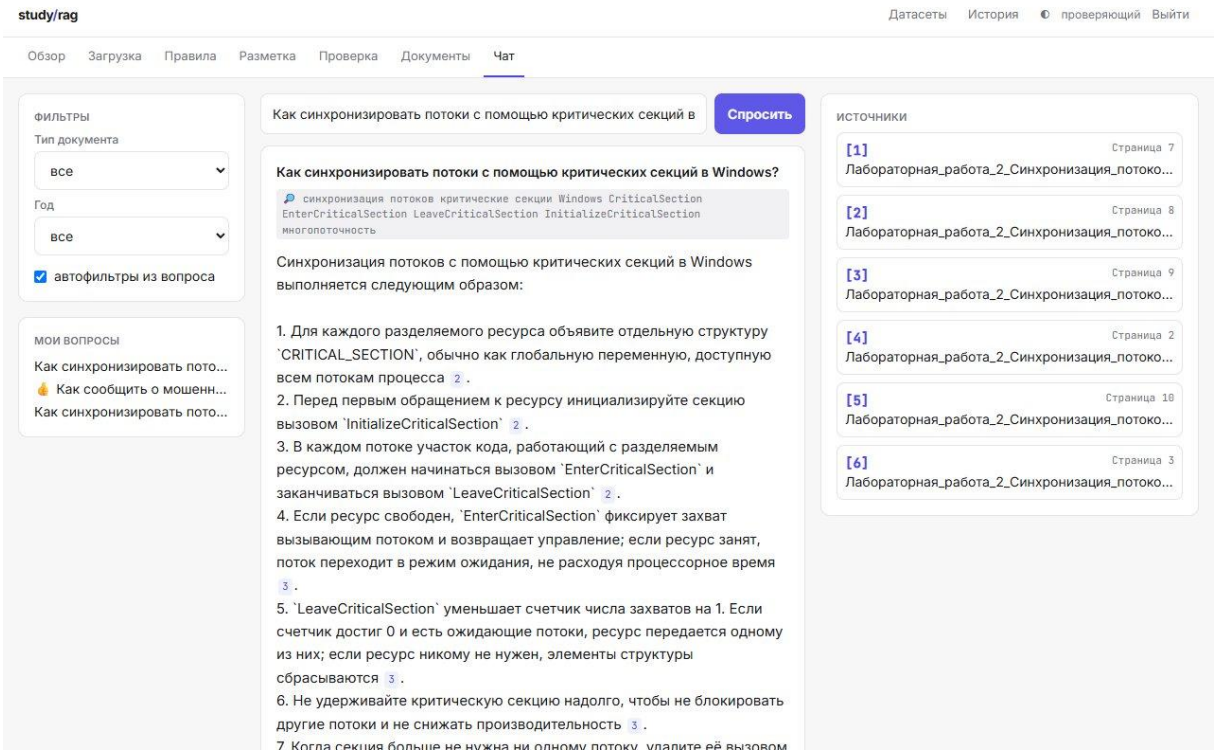


Рис. 14. Чат тестового датасета проверяющего: ответ со сносками, переписанный запрос, источники с номерами страниц

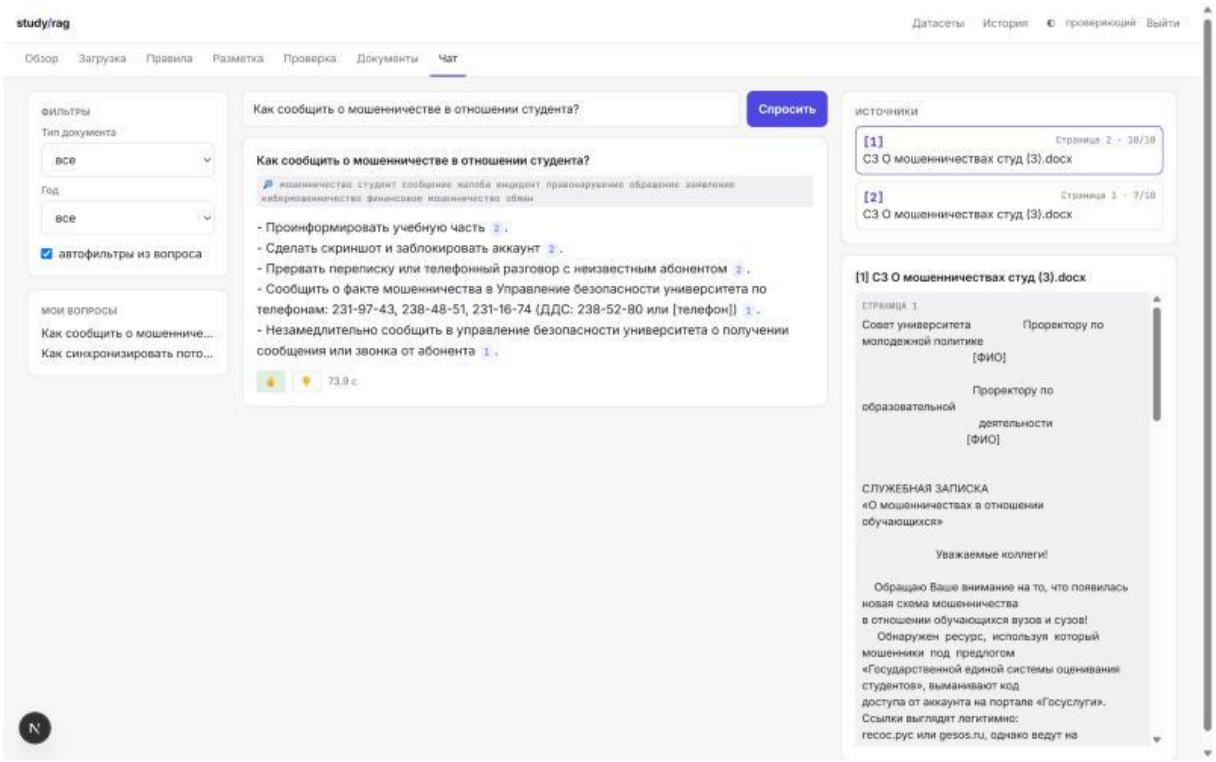


Рис. 15. Панель источника с подсвеченной страницей (вопрос от 08.11.2026)

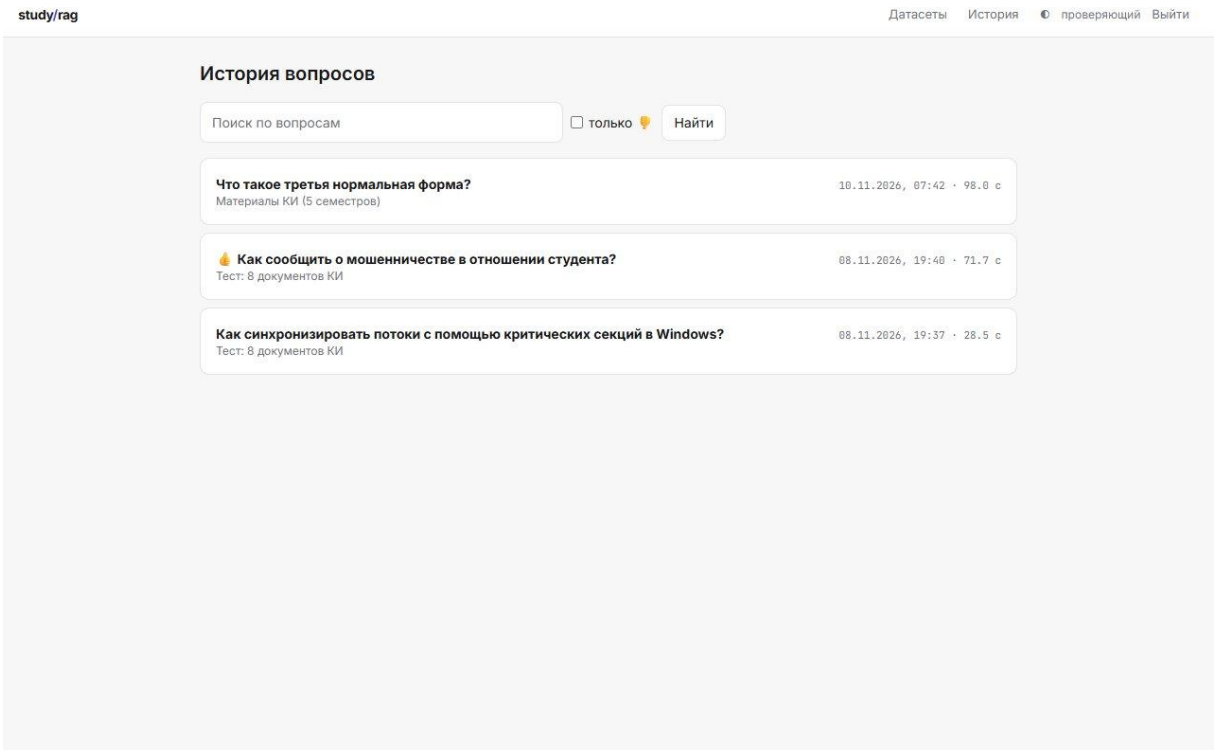


Рис. 16. История вопросов с поиском и фильтром «только 👎»

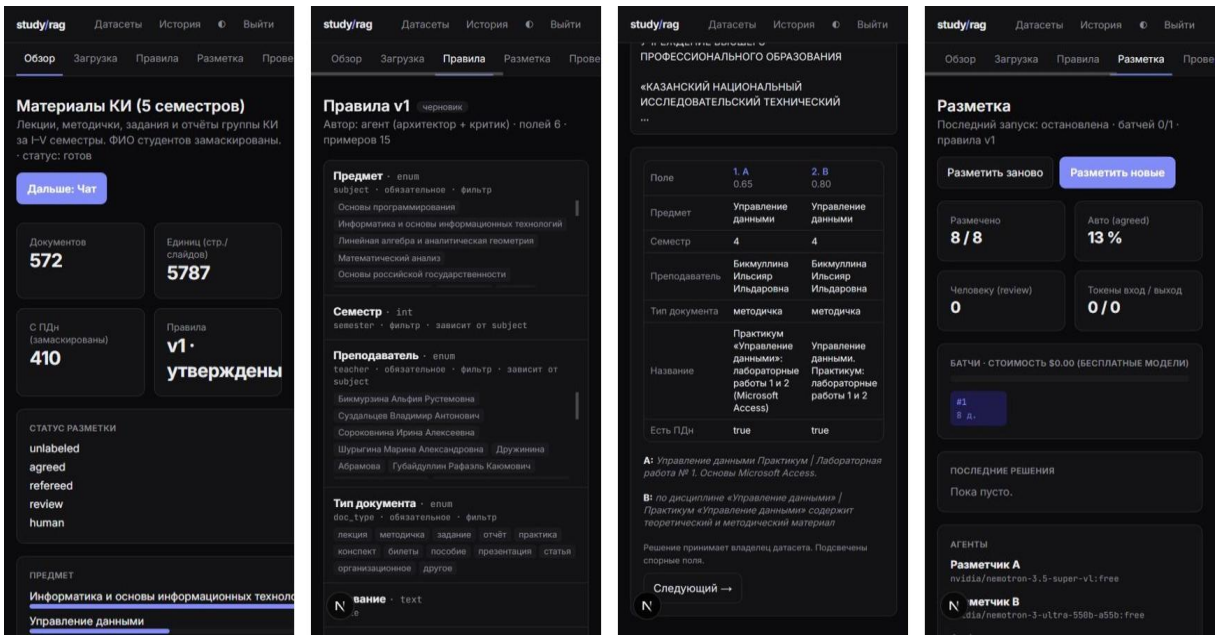


Рис. 17. Мобильная версия (375 px, тёмная тема): обзор, правила, проверка, доска

8.3 Text-to-UI: экраны из v0

Для требования Text-to-UI экраны чата и доски разметки сгенерировал сервис v0 (Vercel) по текстовому описанию. v0 подключился к репозиторию и создал ветку `v0/chat-and-labeling-board` от состояния `main` до этапов 4–7: 15 файлов, 937 строк. Он прочитал `docs/design.md` и сделал экраны в нашем варианте A: чат с фильтрами, ответом со сносками и просмотром источника, доску с KPI, сеткой батчей, дорожками агентов и таблицей решений, а также чистые функции `src/lib/board.ts` и `src/lib/chat.ts`.

Чем экраны v0 отличаются от рабочих, показано в таблице 12.

Таблица 12. Экраны v0 и экраны в main

	Ветка v0	**main**
Ответ на вопрос	маршрут-заглушка, отвечает 501 «генерация ещё не подключена»	SSE-стрим, переписанный запрос, снимаемые автофильтры, оценки rerank, 👍 / 👎
Структура	мелкие компоненты: chat/* и board/* (9 файлов)	один клиентский компонент Chat.tsx и страница доски
Мобильная версия	не проверяли	вкладки вместо колонок, проверено на 375 px
Предпросмотр	падал без переменных Supabase; v0 сам добавил заглушку в proxy.ts	—

Ветку мы не сливаем в `main`: она создана до этапов 4–7, конфликтует с готовыми экранами, а заглушка в `proxy.ts` нужна только для предпросмотра v0. Вывод: по хорошему текстовому описанию и файлу дизайна v0 быстро даёт аккуратную раскладку экрана, но всё, что требует данных и потоков (стрим ответа, очередь), остаётся заглушкой.

9. Тестирование и CI/CD

9.1 Результаты проверок

Автоматические проверки перечислены в таблице 13.

Таблица 13. Автоматические проверки

Проверка	Количество	Что покрывает
Unit-тесты ядра (Vitest)	202	загрузка и ПДн, схема и валидатор, батчи, слияние, каппа, правила, разметчик, воркер, чанки, поиск, точность
SQL/RLS-проверки (PGlite в памяти)	25	RLS чужих датасетов, лимиты демо, уникальность, очередь claim/finish, права на функции
Smoke по живому серверу	3	health, закрытые страницы и API без входа, демо-вход видит демо-датасет
Линтер и типы	—	ESLint, `next typegen` + `tsc --noEmit`

Всего 227 автотестов в 21 файле, все проходят; smoke-тесты (3) запускаются отдельно по адресу сервера и зелёные на продовой сборке. SQL-тесты гоняют настоящие миграции в PGlite — встраиваемом Postgres в памяти с pgvector, поэтому RLS проверяется без Docker и без доступа к облачной БД.

9.2 Конвейер GitHub Actions

На каждый push в `main` и на pull request выполняются `npm ci`, линтер, генерация типов маршрутов и проверка типов, тесты и сборка. Сборке нужны только публичные адреса, реальные ключи живут в Vercel. Vercel сам собирает и выкладывает каждый коммит в `main`.

9.3 Ручная проверка

Каждый этап заканчивался проверкой во встроенном браузере: десктоп, ширина 375 px, тёмная тема. Такие проверки нашли ошибки, которых не видели тесты: после «Утвердить» страница показывала «черновик» (одновременные `router.push` и `router.refresh`), проверяющий не видел голоса агентов в очереди, на телефоне таблица полей сжимала значения до

слова в строке, доска переставала обновляться раньше, чем записывалась сводка.

10. Развёртывание

10.1 Публичный стенд

Приложение работает на Vercel Hobby: https://study-rag-livid.vercel.app. Вход для проверяющего — кнопка «Войти как проверяющий»: сервер входит демо-пользователем, пароль в браузер не уходит. Проверяющему видны основной датасет (только чтение) и свой тестовый датасет, где доступны загрузка, правила, разметка и проверка. После выкладки smoke-тесты по проду прошли 3 из 3, тик воркера из pg_cron отвечает 202.

10.2 Настройки

Настройки задаются переменными окружения (таблица 14); значения хранятся в Vercel и в локальном `.env.local`, в репозиторий не попадают.

Таблица 14. Переменные окружения

Переменная	**Где используется**
NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY	браузер и сервер, под сессией пользователя
SUPABASE_SECRET_KEY	только воркер и скрипты
OPENROUTER_API_KEY, CLOUDFLARE_ACCOUNT_ID, CLOUDFLARE_API_TOKEN	вызов моделей и эмбеддингов
LLM_MODEL, ARCHITECT_MODEL, LABELER_A_MODEL, LABELER_B_MODEL, REFEREE_MODEL, FAST_MODEL, ASK_MODEL	модели по ролям, запасные — через запятую
EMBEDDING_MODEL	модель эмбеддингов Cloudflare
DEMO_EMAIL, DEMO_PASSWORD	серверный демо-вход
WORKER_SECRET	секрет тика; тот же — в Supabase Vault для pg_cron

10.3 Docker

Dockerfile собирает Next.js в режиме `output: "standalone"` и запускает его на Node 24; проверка — `GET /api/health`. Standalone-сборка и health проверены локально без Docker. Сам `docker build` не проверен: Docker на машине разработчика не установлен.

11. Работа с ИИ-агентами

11.1 Инструменты и ход работы

Основной инструмент — Claude Code (модель Claude Opus 5.5): он писал код, миграции, тесты и документацию. Для объёмной работы он запускал субагентов с чистым контекстом: 29 разметчиков A (Claude Sonnet), 5 арбитров и 8 разметчиков серебряного набора (Claude Opus), критиков тестов и кода (Claude Opus). Разметчик B частично — Codex. Сбор данных — Codex на компьютере Тугулевой К.С. Text-to-UI — v0. Журнал с промптами, проблемами и выводами по каждому этапу — `docs/ai-log.md`.

Чтобы длинная работа не расползалась, держались нескольких правил:

- требования записаны чеклистом `docs/requirements.md` в первый день и сверяются на каждом этапе: в первом варианте плана ИИ пропустил CRUD, Dockerfile и журнал, а RAG предлагал упростить до одного векторного поиска;
- состояние между чатами передаётся через `STATUS.md` (цель, решения, что сделано, что пробовали и не сработало), а новый чат начинает со сверки этого файла с git log;
- Next.js 16 вышел после обучения модели, поэтому перед кодом агент читает документацию из `node_modules/next/dist/docs/` (`proxy.ts` вместо middleware, `cacheComponents`);
- ядра загрузки и разметки сделаны через TDD с двумя критиками в чистом контексте;
- агентам-разметчикам разрешено читать только правила и свой батч, запрещено открывать сырые данные и чужие выходы.

11.2 Что нашли проверки

Самые показательные случаи — в таблице 15.

Таблица 15. Ошибки, найденные критиками и сверкой

Где	Что нашли	Чем грозило
Критик кода, ядро разметки	список преподавателей по префиксу фамилии оставлял открытым студента «Иванченко» при преподавателе «Иванов»	утечка ПДн
Критик кода, ядро разметки	поле `constructor` бралось из прототипа объекта; регулярка после «гр.» перебирала экспоненциально	падение и зависание на чужих данных
Финальный критик, загрузка	правило «9+ цифр — номер документа» дало 2 168 ложных замен в лабах по системам счисления	испорченный корпус
Финальный критик, загрузка	флаг ПДн стоял у всех 572 документов из-за домашней папки сборщика в пути	бесполезный флаг
Сверка с офлайн-подсчётом	счётчик битых символов превратил документ из 8 слайдов в 1	потеря страниц
Сверка отчёта с данными	в черновике `eval.md` «21 вопрос ссылается на имя файла» (на деле 1) и «в 3,5 раза» (на деле в 4); ложных отказов 6 вместо 2	завышенные или неверные выводы

Во всех случаях тесты были зелёными. Ошибки нашлись, когда независимый агент прогонял код на всех 572 реальных документах или когда цифры в тексте сверялись с таблицами.

11.3 Выводы

– ИИ хорошо собирает стек и пишет код под требования, но сам склонен упрощать. Что нельзя упрощать, надо записать в начале и сверяться.
– Независимый критик с прогоном на реальных данных окупился сильнее, чем критик тестов: он находил дефекты, которые портили весь корпус.
– Бесплатные модели годятся для разметки и ответов, но нестабильны по доступности. Цепочка запасных моделей и тайм-ауты на шаг важнее выбора «лучшей» модели.
– Выводы отчёта ИИ иногда округляет в свою пользу; каждую цифру нужно сверять с таблицей, из которой она взята.

12. Ограничения и возможное развитие

12.1 Что не проверено

– Очередь проверки из 99 документов людьми не пройдена; точность разметки оценена по серебряному набору модели и 19 ручным проверкам.
– Настоящих вопросов от студентов нет, оценка поиска — на вопросах, сгенерированных моделью.
– Аналоги сравнены по открытым описаниям, без собственного прогона.
– `docker build` не запускался.

12.2 Известные ограничения

– Ответ бесплатной моделью идёт 30–100 секунд; дневные лимиты — 50 запросов OpenRouter и 10 000 нейронов Cloudflare. Индексация 11 082 фрагментов съела около 8 тыс. нейронов, поэтому переиндексировать большой датасет можно только в день без других задач.
– Rerank бесплатной моделью ухудшает выдачу (−10 п. п. hit@5 на подвыборке).
– 10.11.2026 вопрос к основному датасету упал по тайм-ауту запроса к БД (у роли authenticated в Supabase лимит 8 с). Политика RLS проверяла доступ на каждом из 11 082 фрагментов: поиск под пользователем шёл 1,3–1,8 с против 0,5 с без RLS, а на холодной базе выходил за лимит. Исправлено миграцией 9: функции поиска проверяют доступ к датасету один раз (security definer), поиск под пользователем — 0,45 с, вопрос к основному датасету отвечает (56 с весь путь с бесплатными моделями). Первый запрос к «уснувшей» базе Supabase Free по-прежнему идёт несколько секунд.

– Правила v1 и метки основного датасета сделаны офлайн и утверждены в интерфейсе задним числом; основной датасет загружен и проиндексирован скриптами тем же ядром (572 файла не выбрать во встроенном браузере, индексация дольше лимита функции).
– Все 5 014 картинок без описаний: оригиналов в архиве нет; 104 имени файлов не восстановить.
– Поле «есть ПДн» в правилах v1 понимается не так, как его понимает человек.

12.3 Что можно улучшить

– Правила v2: уточнить «есть ПДн» и семестр по расхождениям с ручной проверкой, исправить противоречивый пример в правилах.
– Пройти очередь проверки и собрать золотой набор людьми; собрать 15–20 настоящих вопросов.
– Rerank: порог 3/10, rerank только для отказа или более сильная модель.
– Загрузить материалы Blackboard вторым пакетом в тот же датасет и разметить по тем же правилам.
– Описания картинок, если вернутся оригиналы слайдов.

Заключение

Сделано работающее веб-приложение Study RAG: оно принимает неразмеченный набор учебных документов, маскирует персональные данные, с помощью агентов придумывает схему и правила разметки, размечает корпус двумя разметчиками с арбитром и очередью человеку и отвечает на вопросы со сносками на документ и страницу. Приложение развёрнуто на бесплатных тарифах, проверено 227 автотестами и smoke-тестами, весь путь можно пройти под демо-входом.

На материалах группы КИ (572 документа) схема «двое + арбитр» отправила человеку 17,3 % документов; на выборке из 80 они расходятся с серебряным набором примерно в 4 раза чаще согласованных. Верный фильтр по полю «предмет» поднял hit@5 поиска с 73,6 до 83,0 % — это цель 0,8, но в пределах погрешности, и при фильтре, заданном заранее, а не угаданном из вопроса. Rerank бесплатной моделью, наоборот, ухудшил выдачу и нуждается в доработке.

Главные ограничения оценки — серебряный набор сделан моделью, вопросы сгенерированы, а весь путь ответа с автофильтром и rerank на всех вопросах не измерен. Следующий шаг — правила v2, ручная проверка очереди и настоящие вопросы студентов: тогда метрики можно будет назвать точностью, а не согласием с моделью.

Список литературы

1. Next.js Documentation [Электронный ресурс]. — URL: https://nextjs.org/docs (дата обращения: 10.11.2026).
2. Supabase Documentation [Электронный ресурс]. — URL: https://supabase.com/docs (дата обращения: 10.11.2026).
3. pgvector: open-source vector similarity search for Postgres [Электронный ресурс]. — URL: https://github.com/pgvector/pgvector (дата обращения: 10.11.2026).
4. PostgreSQL Documentation. Chapter 12. Full Text Search [Электронный ресурс]. — URL: https://www.postgresql.org/docs/current/textsearch.html (дата обращения: 10.11.2026).
5. Lewis P. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks // Advances in Neural Information Processing Systems 33 (NeurIPS 2020). — 2020.
6. Cormack G. V., Clarke C. L. A., Büttcher S. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods // Proceedings of SIGIR 2009. — 2009. — P. 758–759.
7. Chen J. et al. BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation. — arXiv:2402.03216. — 2024.
8. Cohen J. A Coefficient of Agreement for Nominal Scales // Educational and Psychological Measurement. — 1960. — Vol. 20, № 1. — P. 37–46.
9. Wilson E. B. Probable Inference, the Law of Succession, and Statistical Inference // Journal of the American Statistical Association. — 1927. — Vol. 22, № 158. — P. 209–212.
10. OpenRouter Documentation [Электронный ресурс]. — URL: https://openrouter.ai/docs (дата обращения: 10.11.2026).

11. Cloudflare Workers AI Documentation [Электронный ресурс]. — URL: https://developers.cloudflare.com/workers-ai/ (дата обращения: 10.11.2026).
12. PGlite — Postgres in WASM [Электронный ресурс]. — URL: https://pglite.dev (дата обращения: 10.11.2026).
13. NotebookLM Help [Электронный ресурс]. — URL: https://support.google.com/notebooklm (дата обращения: 10.11.2026).
14. Label Studio Documentation [Электронный ресурс]. — URL: https://labelstud.io/guide/ (дата обращения: 10.11.2026).
15. v0 by Vercel [Электронный ресурс]. — URL: https://v0.dev (дата обращения: 10.11.2026).